

Exercise set 1

Linear regression and k-nearest neighbor (KNN) prediction, one predictor

- In simple linear regression the prediction of Y for a given $X = x_0$ is

$$\hat{f}(x) = \hat{\alpha}_0 + \hat{\alpha}_1 x_0$$

where

$$\hat{\alpha}_1 = \frac{n \sum_{i=1}^n x_i y_i - [\sum_{i=1}^n y_i] [\sum_{i=1}^n x_i]}{n \sum_{i=1}^n x_i^2 - [\sum_{i=1}^n x_i]^2}$$

and

$$\hat{\alpha}_0 = \bar{y} - \hat{\alpha}_1 \bar{x}.$$

- In *KNN* with one predictor the prediction of Y for a given $X = x_0$ is the average of the k y_i 's which have the smallest corresponding absolute difference $|x_i - x_0|$.

Consider the data set

	x	y
1	1.00	1.88
2	2.00	4.54
3	3.00	10.12
4	4.00	9.14
5	5.00	11.26

Task 1

- a) What is the prediction for $x = 3$ if the linear regression model is used? Use a calculator (or use R as a calculator) to answer this question.

Solution

```
# The data
x=1:5
y=c(1.88,4.54,10.12,9.14,11.26)
n=length(x)
# Fit model
a1=(n*sum(x*y)-sum(y)*sum(x))/(n*sum(x^2)-sum(x)^2)
a0=mean(y)-a1*mean(x)
# Prediction for x=3
a0+a1*3
```

```
## [1] 7.388
```

- b) Use the `lm`-function in R to fit the same model as in a). Use the `predict`-function to compute the prediction of $x = 3$.

Solution

```
# The data
x=1:5
y=c(1.88,4.54,10.12,9.14,11.26)
```

```
# Fit model
reg1=lm(y~x)
# Prediction for x=3
predict(reg1,data.frame(x=3))
```

```
##      1
## 7.388
```

- c) What is the prediction for $x = 3$ if KNN with $K = 3$ is used? For $K = 1$ and $K = 5$? Use a calculator (or use R as a calculator) to answer this question.

Solution

```
# KNN for K=3
(4.54+10.12+9.14)/3
```

```
## [1] 7.933333
```

```
# KNN for K=1
10.12
```

```
## [1] 10.12
```

```
# KNN for K=5
mean(y)
```

```
## [1] 7.388
```

- d) Consider the R-function

```
knn=function(x0,x,y,K=20)
{
  d=abs(x-x0)
  o=order(d)[1:K]
  ypred=mean(y[o])
  return(ypred)
}
```

Experiment with R to see and explain what happens in each row of the function if $x_0 = 3$ and $K = 3$. Start by

```
K=3
x0=3
x=1:5
y=c(1.88,4.54,10.12,9.14,11.26)
d=abs(x-x0)
d
```

```
## [1] 2 1 0 1 2
```

Solution

```
K=3
x0=3
x=1:5
y=c(1.88,4.54,10.12,9.14,11.26)
d=abs(x-x0)
d # Distances between x0 and the different x-values
```

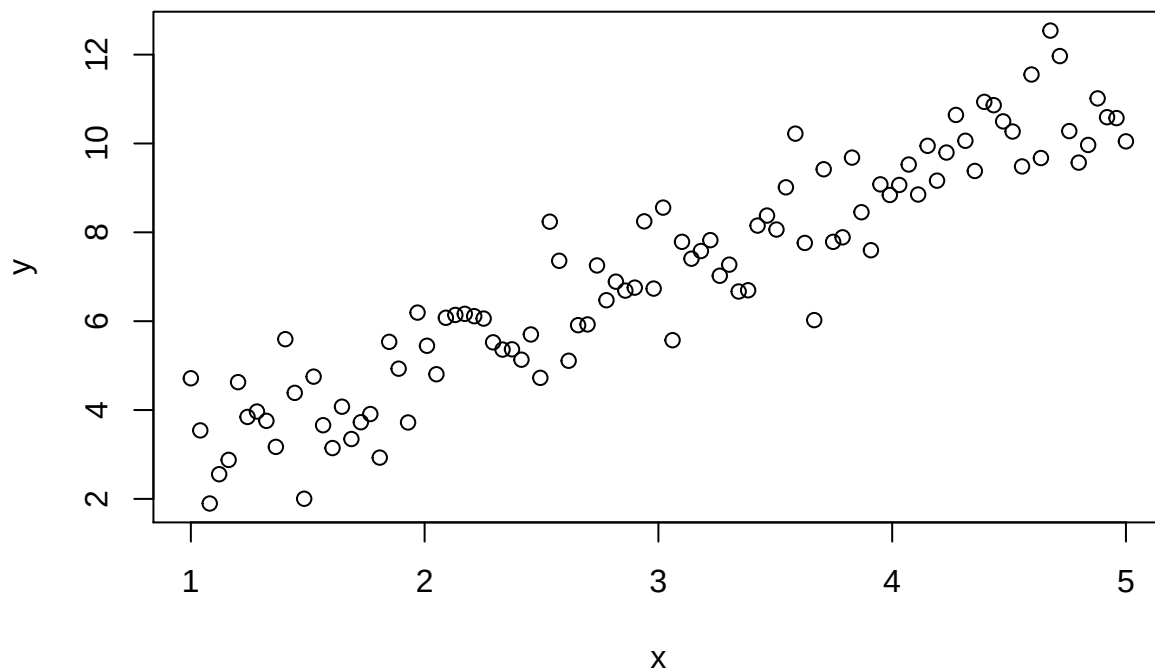
```
## [1] 2 1 0 1 2
```

```
o=order(d)[1:K]
o # Indices of the three smallest distances

## [1] 3 2 4
ypred=mean(y[o])
ypred # Average of the y's for the 3 observations with smallest d

## [1] 7.933333
# return(ypred) returns ypred when the function is called
```

- e) Which of the 4 methods, linear regression or KNN with $K=1,2$ or 3 , would you use if you had seen the following plot? Why?



Solution Linear regression. It is plausible that this relationship is linear and if it is, this assumption the data will efficiently be used to estimate two parameters for the purpose of predicting Y for any X .

Test and training data

- *Training data* is data which is used to fit the model.
- *Test data* is data which is used to test the predictions of the model (and not used to fit the model).

Task 2

- a) Load, summarize, and read the help to the dataset College from the R-package ISLR by

```
library(ISLR)
summary(College)
```

```
## Private      Apps      Accept      Enroll      Top10perc
## No :212      Min.      : 81      Min.      : 72      Min.      : 35      Min.      : 1.00
## Yes:565      1st Qu.: 776      1st Qu.: 604      1st Qu.: 242      1st Qu.:15.00
##              Median : 1558      Median : 1110      Median : 434      Median :23.00
##              Mean   : 3002      Mean   : 2019      Mean   : 780      Mean   :27.56
##              3rd Qu.: 3624      3rd Qu.: 2424      3rd Qu.: 902      3rd Qu.:35.00
##              Max.   :48094      Max.   :26330      Max.   :6392      Max.   :96.00
## Top25perc    F.Undergrad    P.Undergrad    Outstate
## Min.      : 9.0      Min.      : 139      Min.      : 1.0      Min.      : 2340
## 1st Qu.: 41.0      1st Qu.: 992      1st Qu.: 95.0      1st Qu.: 7320
## Median : 54.0      Median : 1707      Median : 353.0      Median : 9990
## Mean   : 55.8      Mean   : 3700      Mean   : 855.3      Mean   :10441
## 3rd Qu.: 69.0      3rd Qu.: 4005      3rd Qu.: 967.0      3rd Qu.:12925
## Max.   :100.0      Max.   :31643      Max.   :21836.0      Max.   :21700
## Room.Board    Books      Personal      PhD
## Min.      :1780      Min.      : 96.0      Min.      : 250      Min.      : 8.00
## 1st Qu.:3597      1st Qu.: 470.0      1st Qu.: 850      1st Qu.: 62.00
## Median :4200      Median : 500.0      Median :1200      Median : 75.00
## Mean   :4358      Mean   : 549.4      Mean   :1341      Mean   : 72.66
## 3rd Qu.:5050      3rd Qu.: 600.0      3rd Qu.:1700      3rd Qu.: 85.00
## Max.   :8124      Max.   :2340.0      Max.   :6800      Max.   :103.00
## Terminal      S.F.Ratio      perc.alumni      Expend
## Min.      : 24.0      Min.      : 2.50      Min.      : 0.00      Min.      : 3186
## 1st Qu.: 71.0      1st Qu.:11.50      1st Qu.:13.00      1st Qu.: 6751
## Median : 82.0      Median :13.60      Median :21.00      Median : 8377
## Mean   : 79.7      Mean   :14.09      Mean   :22.74      Mean   : 9660
## 3rd Qu.: 92.0      3rd Qu.:16.50      3rd Qu.:31.00      3rd Qu.:10830
## Max.   :100.0      Max.   :39.80      Max.   :64.00      Max.   :56233
## Grad.Rate
## Min.      : 10.00
## 1st Qu.: 53.00
## Median : 65.00
## Mean   : 65.46
## 3rd Qu.: 78.00
## Max.   :118.00
```

```
help(College)
```

b) Divide the data into a 50/50 training and test data by

```
set.seed(123)
n=nrow(College)
train_indicator=sample(1:n,size=floor(n/2))
train=College[train_indicator,]
test=College[-train_indicator,]
```

c) Fit a linear prediction model, on the training data, for number of applications, Apps, with the predictors Private and Accept by

```
m1=lm(Apps~Private+Accept,data=train)
summary(m1)
```

```
##
## Call:
## lm(formula = Apps ~ Private + Accept, data = train)
##
```

```
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2776.2  -354.8  -149.2    13.3   7869.2
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -154.26497   171.42683  -0.900   0.369
## PrivateYes   129.79797   157.99418   0.822   0.412
## Accept       1.52692     0.03556  42.940 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1204 on 385 degrees of freedom
## Multiple R-squared:  0.8638, Adjusted R-squared:  0.8631
## F-statistic: 1221 on 2 and 385 DF,  p-value: < 2.2e-16
```

d) Compute training-MSE by

```
pred_train=predict(m1)
mse_train=mean((train$Apps-pred_train)^2)
mse_train
```

```
## [1] 1437492
```

e) Now, redo c) and d) but by only using Accept as the predictor. Fit the model and compute the training-MSE. Is it larger or smaller?

Solution

```
m2=lm(Apps~Accept,data=train)
pred_train2=predict(m2)
mse_train2=mean((train$Apps-pred_train2)^2)
mse_train2
```

```
## [1] 1440012
```

It is larger which is natural since the model is fitted with one regressor less than in c) and d). This is equivalent with the fact that the R^2 for a linear regression always increases with the number of X -variables, regardless if they are relevant or not.

f) Finally compute the test-MSE for the two regression models fitted in c) and e). This is how you do it for the one in c)

```
pred_test=predict(m1,newdata = test)
mse_test=mean((test$Apps-pred_test)^2)
```

Solution

```
mse_test
```

```
## [1] 1849668
```

```
pred_test2=predict(m2,newdata = test)
mse_test2=mean((test$Apps-pred_test2)^2)
mse_test2
```

```
## [1] 1854227
```

The model with both **Private** and **Accept** gave more precise test-predictions. This is not necessarily the case. A small model can very well give a smaller **testMSE** than a large model; a contrast with **trainingMSE**.

- g) Comment on the relationships between the MSE's between the models in trainingset and testset respectively.

Solution The `testMSE` is larger for both models than the `trainingMSE`. This is not surprising since when fitting a linear regression, the `trainingMSE` is minimized.

- h) Use KNN to predict Apps. Compare the test-MSE with the linear regression predictions.

Solution

```
knn=function(x0,x,y,K=3)
{
  dist=abs(x-x0)
  odist=order(dist)
  y0=mean(y[odist[1:K]])
  return(y0)
}
x0=test$Accept
x=train$Accept
y=train$Apps
pred_test3=apply(as.matrix(x0),1,knn,x=x,y=y)
mse_test3=mean((test$Apps-pred_test3)^2)
mse_test3
```

```
## [1] 4473360
```

We could experiment with different values of $K = 3$ but it seems difficult to beat the simple linear regression with `Accept` as the regressor.

Task 3

Do Exercise 3.10a-f in the book and, also, do the extra task

- g*) Compare the predictions by splitting the data into a training and a test part, 50% in each.

```
# Read data
library(ISLR)

# First look at data
help(Carseats)
head(Carseats)
```

```
##   Sales CompPrice Income Advertising Population Price ShelfLoc Age Education
## 1  9.50      138     73          11         276    120      Bad   42         17
## 2 11.22      111     48          16         260     83     Good   65         10
## 3 10.06      113     35          10         269     80   Medium   59         12
## 4  7.40      117    100           4         466     97   Medium   55         14
## 5  4.15      141     64           3         340    128     Bad   38         13
## 6 10.81      124    113          13         501     72     Bad   78         16
##   Urban  US
## 1   Yes Yes
## 2   Yes Yes
## 3   Yes Yes
## 4   Yes Yes
## 5   Yes  No
## 6   No  Yes
```

```
class(Carseats)
```

```
## [1] "data.frame"
```

```
supply(Carseats,class)
```

```
##      Sales    CompPrice      Income Advertising Population      Price
## "numeric"  "numeric"    "numeric"  "numeric"  "numeric"  "numeric"
## ShelfLoc      Age    Education      Urban      US
## "factor"    "numeric"  "numeric"    "factor"    "factor"
```

```
# (a)
```

```
reg1=lm(Sales~Price+Urban+US,data=Carseats)
```

```
# (b)
```

```
summary(reg1)
```

```
##
## Call:
## lm(formula = Sales ~ Price + Urban + US, data = Carseats)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -6.9206 -1.6220 -0.0564  1.5786  7.0581
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 13.043469   0.651012  20.036 < 2e-16 ***
## Price       -0.054459   0.005242 -10.389 < 2e-16 ***
## UrbanYes    -0.021916   0.271650  -0.081  0.936
## USYes       1.200573    0.259042   4.635 4.86e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.472 on 396 degrees of freedom
## Multiple R-squared:  0.2393, Adjusted R-squared:  0.2335
## F-statistic: 41.52 on 3 and 396 DF, p-value: < 2.2e-16
```

```
# Price: Everything else equal, if price increases with one, on average
#         54 less carseats will be sold
# Urban: Everything else equal, if the store is in an urban location,
#         on average 22 less carseats will be sold compared to a rural location
# US:    Everything else equal, on average, 1200 more carseats is sold in US stores than
#         in other stores.
# Inter: If all explanatory variables are zero, on average, 13043 carseat is sold
```

```
# (c)
```

```
# Sales=13.04-0.055 Price-0.022 Urban+1201US+epsilon
# where Urban is one for urban location and zero otherwise,
# US is one for US stores and zero otherwise and
# epsilon is a stochastic error term with expectation zero and
# standard deviation 2.472
```

```
# (d)
```

```
# H0: beta_Price=0
# H0: beta_Price not=0
# p-value approximately zero. We reject H0 on almost any significance level.
# H0: beta_Urban=0
# H0: beta_Urban not=0
```

```
# p-value=0.989. We do not reject H0 on standard significance levels (0.05,0.01,...).
# H0: beta_US=0
# H0: beta_US not=0
# p-value=0.0000049. We reject H0 on significance levels larger than 0.000005.
```

```
# (e)
```

```
reg2=lm(Sales~Price+US,data=Carseats)
summary(reg2)
```

```
##
## Call:
## lm(formula = Sales ~ Price + US, data = Carseats)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -6.9269 -1.6286 -0.0574  1.5766  7.0515
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  13.03079    0.63098   20.652 < 2e-16 ***
## Price        -0.05448    0.00523  -10.416 < 2e-16 ***
## USYes         1.19964    0.25846   4.641 4.71e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.469 on 397 degrees of freedom
## Multiple R-squared:  0.2393, Adjusted R-squared:  0.2354
## F-statistic: 62.43 on 2 and 397 DF,  p-value: < 2.2e-16
```

```
# R-squared in a: 0.2393
# Adjusted R-squared in a: 0.2335
# R-squared in e: 0.2393
# Adjusted R-squared in e: 0.2354
```

```
# Extra: Compare by training/test data (50/50)
```

```
n=nrow(Carseats)
ntest=n/2
set.seed(12345)
ind=sample(1:n,size=ntest)
traindata=Carseats[ind,]
testdata=Carseats[-ind,]
reg3=lm(Sales~Price+Urban+US,data=traindata)
pred3=predict(reg3,newdata=testdata)
mse3=mean((testdata$Sales-pred3)^2)
mse3
```

```
## [1] 6.662185
```

```
reg4=lm(Sales~Price+US,data=traindata)
pred4=predict(reg4,newdata=testdata)
mse4=mean((testdata$Sales-pred4)^2)
mse4
```

```
## [1] 6.580072
```